

Scalable Modularised MLOps with Quality Framework

Peeyush Pashine*
Abhishek Sengupta†
Ashwini Chandrashekharaiiah
Jayit Saha
Vijay Pandey
Ganesh Singh
Walmart Global Tech

ABSTRACT

Abstract: As DS (Data Science) based products stabilize, evolve, and broaden in functionality, scope and scale, there is a need for a robust MLOps (Machine Learning Operations) framework that can support the scaling and productization of DS modules. The framework should provide automation, capabilities to run with minimal intervention and optimize the time and cost required to run the DS Modules in production. As the scale of the product increases, it makes quality (data and model) control challenging, thus, the framework should be supported by an end-to-end Quality Assurance and Enhancement (QAE) Pipeline to help retain the quality standards. In this paper, we present a novel MLOps framework coupled with a QAE Pipeline that incorporates standard DevOps principles and introduces new practices specific to Data Science.

KEYWORDS

MLOps, Scale for growth, Code quality, Modularisation

1 INTRODUCTION

The process of transforming interconnected data science modules into a product is very challenging which necessitates the need for standardization of code structure across all the modules. Additionally, processes like integration, testing, code versioning and reviews need to be well-defined and automated to provide a seamless product experience. To support the scale, a hybrid machine learning approach needs to be established that combines DS modules with scalable tech stacks. Also, a framework that supports quality assurance of such pipelines, is critical to maintain overall output quality.

In this paper, we talk about a NOvel QUALity assurance, enhancement and MLOps framework at Scale (NO-QUALMS) which provides an end-to-end automation of the data science lifecycle and ensures reproducibility and traceability. The framework promotes better collaboration and communication among the teams working on the project. It also addresses the challenge of scaling up data science modules by using standardized templates, built with a combination of native PySpark APIs and custom user-defined functions (UDFs).

The automated Quality Assurance pipeline ensures quality excellence throughout all stages of model development, starting from data preparation to model deployment. The framework includes various components such as Data Health Reports, Quality Improvement through ML, Bias Detection Modules, Model Maintenance, Drift Detection and Correction. The setup minimizes the need for manual checks via automated alerts, in case the required quality

standards are not met, thereby reducing the risk of bugs, and increasing the reliability of the models.

The framework also incorporates optimization strategies like supporting local development, connections to cloud storage and databases.

2 BACKGROUND

A machine learning pipeline should be capable of handling large volumes of data, and the interface design should be use-case agnostic (batch or real time). The scale is even more significant for machine learning models in Walmart, where solutions parse vast amounts of data from multiple departments, and product hierarchies across regions, channels, and brands. Furthermore, adding a temporal dimension to the analysis significantly increases the time and space complexities.

The three areas majorly addressed in this paper are:

- Speed of development, integration, and deployment.
- Cost and Performance optimization techniques.
- Quality framework and processes.

Rapid feature development involves implementing performance optimization strategies (Section 3.1) across different data science modules to ensure that the pipeline operates within acceptable cost and time thresholds. Further, Modularization techniques (Section 3.2) should be in place for seamless Integration of new modules in the ML pipeline.

To reduce overall costs, Local PySpark setup for development at scale (Section 3.3) has been introduced, along with testing strategies (Section 3.4) to validate DS module outputs.

To ensure quality standards, the solution requires a robust, scalable quality (data and model) control mechanism (Section 3.5), as the size of the data and models makes it impossible to conduct exhaustive manual validations. This, coupled with an ML based Quality Enhancement framework ensures uninterrupted product delivery even in scenarios where the input data falls short of the quality standards.

No-QUALMS automates and packages all the above modules together to form a robust and replicable structure (Fig. 1) which can be reused (with minor tweaks) across any generic Data Science Product.

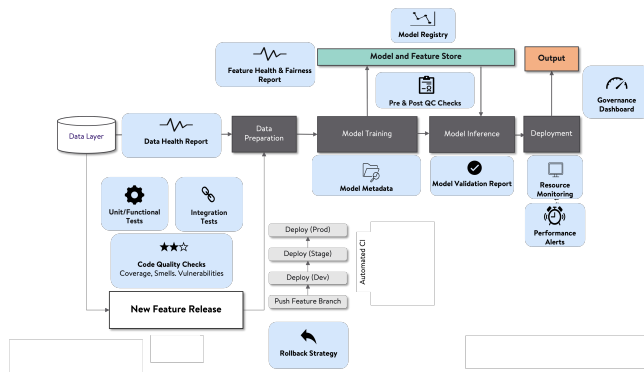


Figure 1: No-QUALMS Framework

3 IMPLEMENTATION

In this section techniques on scaling, optimization, modularization, integration patterns and module testing are highlighted. Additionally, pipeline automation and QAE framework are described in detail.

3.1 Scaling and Optimization

The below points are elaborated further in this subsection:

- Configuration and parameterisation.
- Design pattern/code template.
- Tool for job scheduling.

As mentioned in Section 2, operating across multiple suppliers at various levels of temporal product hierarchy poses a significant challenge. To handle this scale, a distributed computing framework in PySpark [1], which includes a combination of native Spark and Python UDFs, is used in both the Data and Model Pipelines. Best practices of OOPS, Design patterns, and MLOps have been employed to support the scale and future updates. This architecture enables the system to ingest 1 billion rows (8 TB) and infer from 50,000 models at all hierarchies simultaneously within a runtime of 1.5 hours.

The pipeline is designed to execute for all product and time hierarchies by creating a unique index that performs a “Distributed GroupBy” operation, to distribute the data among different executors [2]. To ensure scalability, the solution employs an on-demand cloud provider cluster, leveraging workflow management service to schedule workflows for generating alerts. Fig. 2, 3 and 4 provides an overview of the workflow at scale.

3.2 Modularisation and Automation

To enhance the modularity and scalability of the pipeline, a modern packaging structure has been utilized through build and setup tools such as `setup.py` and `Makefile`, which helps in packaging.

Furthermore, the entire code versioning is essentially managed via code versioning system. An end-to-end parameterization of the entire code and pipeline is supported using a default `config.py`

file, comprising of variables, dictionaries, and queries executed and utilized by all the various modules.

To automate the deployment process, a Continuous Integration/Continuous Deployment (CI/CD) pipeline has been established. The pipeline is built on internal continuous integration and build tools, and container service platform is used as a Deployment Artefact, enabling the deployment of new code changes in a streamlined and efficient manner, with minimal manual intervention.

3.3 Local setup for development

We support instructions and processes around establishing local setup for development, so that data science can leverage local machines without spinning up the clusters for POCs and development [3]. This helps both in cost reduction as well as speeding up the development process [4]. Fig. 2 depicts the flow of development to production both from local and remote machines aspect.

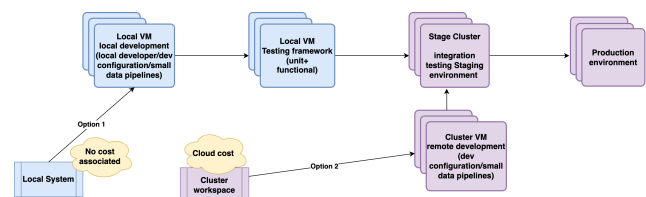


Figure 2: Local and remote development workflow

3.4 Integration and Testing

To ensure the quality of each Data Science module in the pipeline, “unit” (checks for functionality of individual methods in the code) and “functional” (focuses on the output of the module) testing are conducted to ensure that the module meets the desired requirements. Once the module passes these tests, it is passed through a set of “integration” tests before it can be integrated into the development code.

As an example, Fig. 3 illustrates the intermediate output from an anomaly detection engine (Top Table) which is then fed into a new Template Engine to obtain the final textual representation (Bottom Table) of the anomaly. The entries highlighted in red are erroneous entries which have been detected through the unit and functional tests defined as below:

Unit Tests:

- Predicted sales, confidence scores are non-negative.
- Ranking columns are unique.

Functional Tests:

- Outputs in sync with each other.

week	mds_fam_id	Predicted Sales	Confidence Score	Is Anomalous	Current Week Ranking
202246	172231934	3.5	60%	1	1
202246	127368092	2.6	75%	1	2
202246	1098780765	-1.1	120%	1	2



week	mds_fam_id	Insight	Details	Severity Rank
202245	172231934	NULL	Expected Sales drop over 20% for item	NA
202245	1098780765	Potential Sales Drop	Expected Sales rise over 10% for item	NA
202246	172231934	Potential Sales Drop	Expected Sales drop over 20% for item	2
202246	9856753	Potential Sales Drop	Expected Sales drop over 20% for category	1

Figure 3: Integration and testing workflow in ML pipeline.

- Persistent Alerts should be ranked higher.

Additionally, integration tests are performed to ensure that the new Template Engine does not break the code stability.

4 QUALITY ASSURANCE ENHANCEMENT PIPELINE

As mentioned in Section 2, the scale of the solution necessitates a robust scalable quality control mechanism for data (at all levels of the entire ML journey) as well as for all the ML Model. The below sections highlight how an end-to-end Quality Assurance and Enhancement Pipeline has been created at each step to help maintain the quality standards.

4.1 Data Quality Assurance

The Below Key Metrics are calculated for each data source to ensure baseline quality adherence:

- Completeness: missing values in a record column or table

$$q_i = \sum_{j=1}^K q_{ij}; q_{ij} = 1 \text{ if } v_{ij} \text{ is not NULL}$$

$$\text{Completeness} = \frac{\sum_{i=1}^N B(q_i = K)}{N}$$

- Accuracy: closeness of the values the accepted values in a specific domain

$$q_i = \sum_{j=1}^K q_{ij}; q_{ij} = 1 \text{ if } v_{ij} \text{ is syntactically accurate}$$

$$\text{Accuracy} = \frac{\sum_{i=1}^N B(q_i = K)}{N}$$

- Timeliness: Indicates the freshness of the data for a specific application

$$\text{Timeliness} = \max\{0, 1 - \frac{T}{F}\}$$

- Consistency: Indicates level of compliance between two or more columns

$$q_i = \sum_{j=1}^K q_{ij}; q_{ij} = 1 \text{ if } v_{ij} \text{ does not violate constraints}$$

$$\text{Consistency} = \frac{\sum_{i=1}^N B(q_i = K)}{N}$$

- Uniqueness: Captures the number of duplicated records

$$q_i = 0 \text{ if record } i \text{ is not unique}$$

$$\text{Uniqueness} = 1 - \frac{\sum_{i=1}^N B(q_i = 0)}{N}$$

where,

- B= 1 if condition is met, else 0
- v_{ij}: value at col. j and row i
- T: Age of the data
- F: Frequency of the data.

Additional QA checks are performed in the intermediate tables based on additional constraints such as:

- Business Requirements (expose values only beyond a certain threshold)
- Domain Knowledge (non-negative p-values)
- The Model outputs also undergo a series of QA tests categorized into the two below buckets:
 - Pre-Train Tests: (This tests are performed before model parameter tuning)
 - Shape Consistency: Check for output dimensions at each stage
 - Data Format Consistency: Checks for correct Data Formats
 - Data Leakage Checks: No training for future data /testing on past data
 - Post-Train Tests (This tests are performed post model parameter tuning):
 - Invariance Tests: Model Consistency checks by tweaking one feature (E.g. changing gender should not affect individual's eligibility for loan)
 - Directional Expectations: Directional stability between Predictor Coefficients and Response (E.g. having a higher credit score should increase loan eligibility)

PyDeequ [5] is used for conducting all the above Data Quality checks to arrive at an overall Data Health Report, with alerts triggered if the Report “fails” based on pre-defined thresholds.

As an example, in Fig 4, it shows process related to part of QA checks and in Fig 5 the complete QA Report for the table has Completeness and Timeliness values below a pre-defined threshold, which if not detected would have resulted in incomplete (Low Completeness) and older output (Low Timeliness Score) to be generated.

omni_dept_nbr	omni_catg_nbr	mds_fam_id	sales	Sales deviation
46	12529	172231934	200	3.5
46	12509	127368092	120	2.6
72	NULL	1098780765	360	1.1
72	76546	172231923	56	4.2
72	NULL	872923098	231	2.4

check	check_level	check_status	constraint	constraint_status
Integrity checks	Error	Error	SizeConstraint(Si...	Success
Integrity checks	Error	Error	MinimumConstraint...	Failure
Integrity checks	Error	Error	CompletenessConst...	Failure
Integrity checks	Error	Error	UniquenessConstra...	Success
Integrity checks	Error	Error	ComplianceConstra...	Success
Integrity checks	Error	Error	ComplianceConstra...	Success

Figure 4: Process for QA checks

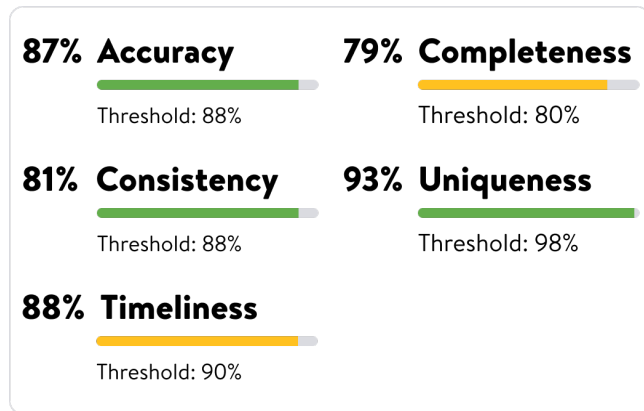


Figure 5: Final QA Report for Data Table

Apart from Quality KPIs, the Modelling data is also checked for Data Fairness, wherein What-if Tool [6] and statistical models are used to respectively check and correct for the below types of biases:

- Sampling Bias: Occurs when the Data Subset is not Representative of the Population.
- Corrective Measure: Use Stratified Sampling for proportionate representation from each Strata.
- Exclusion Bias: Occurs when features seemingly deemed irrelevant are removed prior to Model Building.
- Corrective Measure: Use a low-cost Vanilla Model to Compute Feature Importance before removing features.
- Confounding Bias: Occurs due to confounders in the dataset (correlated variables with respect to both predictor and response)
- Corrective Measure: Partial Correlation Tests (VIFs) to detect and remove multicollinearity.

4.2 Data Quality Enhancement

For instances where the Data Quality Metrics fall below the accepted threshold, Machine Learning Models are used to improve

the Data Quality by imputing missing features, correcting erroneous values, and cleaning duplicated values. Below are the key approaches used to improve the Data Quality through ML:

- Heuristic Approaches: Data Standardizing, Imputation with Mode
- Statistical Measures: Outlier Detection and Removal, MICE (Multiple Imputation using Chained Equations)
- Distance Based Measures: Cosine Similarity Matching, Levenshtein Distance Based Measures for correcting misspelled descriptions, erroneous abbreviations, etc. (Figure 6)

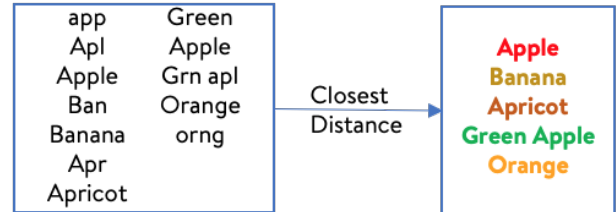


Figure 6: Corrections using Similarity Matching

- Deep Learning Approaches: Word embeddings (TF-IDF, GloVe) and Multi-class classification models for filling in missing text (dept name, ingredients, etc) using other available fields (Fig. 7)

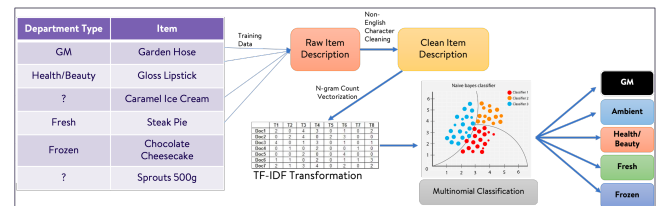


Figure 7: Dept Identification through embeddings

4.3 Model Quality Assurance

Along with data, the quality of the model also needs to be monitored consistently to prevent degradation of model predictive performance resulting in a loss of accuracy.

For model monitoring, performance metrics like Model Accuracy, Precision, Recall, AUC are monitored over a rolling time window and if these metrics fall below a certain threshold (indicating model degradation) the model is re-evaluated and fine-tuned to bring it up to the desired performance. All the above metrics are tracked through MLFlow [7] and converted into a Model Validation Dashboard using Evidently (Fig 7).

Drift is also a common phenomenon experienced in temporal models, wherein the distribution of the input/output data changes, affecting the model's decision boundary and performance metrics. Alibi Detect [8] is used to detect such drifts (Fig. 8) through chi-squared tests (for categorical data) and Maximum Mean Discrepancy Tests [9] for numeric data; and in case of a drift, the model is retrained (with increased weightage to the newer observations) to course-correct for the drift.

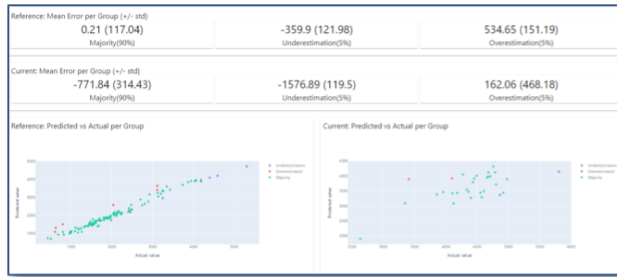


Figure 8: Model Validation Dashboard

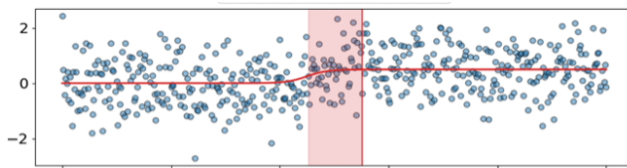


Figure 9: Drift Detection using Alibi Detect

5 IMPACT

No-QUALMS has been piloted for Channel performance DS Insights, which is launched for 50 suppliers with 20K ML models churning data across 40 M item-store combinations on a weekly basis, and it is expected to increase by 200

Through this framework, the development and production costs have reduced by 6x and 3x per month respectively. The QA pipeline has managed to reduce the average time spend in quality assurance to one-third by automating most of the manual checks required.

Figure 12 shows cost optimization impact for Channel performance DS insights.

6 CONCLUSION

In this paper, we have presented NO-QUALMS, an end-to-end Quality Assurance, Enhancement and MLOPS framework for supporting Data Science Modules at Scale. This framework serves as a foundational template for others who wish to build their own QA and MLOps framework as it offers a scalable solution for the operationalization of ML in data science-based product development. This structure is currently being replicated into all other DS products in development within Data Ventures.

7 CITATIONS AND BIBLIOGRAPHIES

[11] [7] [1] [12] [4] [5] [9] [10] [3] [8] [6] [2]

8 ACKNOWLEDGMENTS

We thank Shirish Tatikonda, Ravishankar Balasubramaniam and Vinay Artham for helping out reviewing and suggesting materials in the paper.

REFERENCES

[1] S. H. Bhasha. [n. d.]. Analyzing Optimizing Spark Performance. ([n. d.]).

- [2] B. Caffey. 2020. *Cost Efficient Executor Configuration for Apache Spark*. <https://medium.com/expedia-group-tech/part-3-efficient-executor-configuration-for-apache-spark-b4602929262>.
- [3] R. R. Chindu. 2020. *8 Performance Optimization Techniques Using Spark*. <https://www.syntelli.com/eight-performance-optimization-techniques-using-spark>.
- [4] J. P. M. e. a. Wexler. 2019. The what-if tool: Interactive probing of machine learning models. *IEEE transactions on visualization and computer graphics* (2019).
- [5] M. e. a. Zaharia. 2018. Accelerating the machine learning lifecycle with MLflow. *IEEE Data Eng* (2018).
- [6] Google. 2023. *MLOps: Continuous delivery and automation pipelines in machine learning*. <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>.
- [7] S. Kambhampati. [n. d.]. Explore best practices for Spark performance optimization., ([n. d.]).
- [8] P. Paolo. [n. d.]. *Apache Spark Optimization Techniques*.
- [9] SeldonIO. [n. d.]. *alibi-detect*. <https://github.com/SeldonIO/alibi-detect>
- [10] T. Viehmann. 2021. Partial Wasserstein and Maximum Mean Discrepancy distances for bridging the gap between outlier detection and drift detection. *arXiv* (2021).
- [11] F. Vázquez. 2018. *How to use PySpark on your computer*. <https://towardsdatascience.com/how-to-use-pyspark-on-your-computer-9c7180075617>.
- [12] Spark website. 2018. *Spark Installation*. https://spark.apache.org/docs/latest/api/python/getting_started/install.html. <https://github.com/aws-labs/python-deequ>, 2021.